

Artificial Intelligence 2

- Übungsblatt 1 -

**Institut für Softwaretechnologie
Inffeldgasse 16b/2
A-8010 Graz
Austria**

Individual Exercises (5 P)

- Visit <https://checkr.tugraz.at/> (a TU Graz software, login using your TU Graz account).
- At <https://checkr.tugraz.at/>, enter the key 8ydg5H
- Provide your answers to the posed questions - several tries are possible (20 mins waiting time)!
- If all questions have been answered correctly, this corresponds to 5 points (each correctly answered question corresponds to 0.25 points).
- **No explicit „Abgabe“ is needed for this part (just answer the questions in KnowledgeCheckR)!**

Group Exercises (15P)

Exercise 1.a)

Given the CSP $x_1[1..4]$, $x_2[1..3]$, $c_1:x_1 > x_2$, $c_2:x_2 = 3$. Show the solution search with node-consistency checking + backtracking + forward checking. Implement this example in Minizinc and let Minizinc (www.minizinc.org/) find a solution (5P)

Additional Information:

- You have to execute the “backtracking with forward-checking” algorithm (as described in the course material) step by step.
- Generally, you're free to choose whether you'll start assigning values to x_1 or x_2 , and in what order to assign the values (any permutation of $\{1, 2, 3, 4\}$). If you choose anything other than the “defaults”, give some reasoning why, that is, if you implemented that algorithm, why it would choose that specific variable ordering rather than the default one.
- It's enough to execute the “backtracking with forward checking” algorithm, you don't need to perform the pure (stupid, exponential) backtracking algorithm.
- Minizinc is free/libre software. You can install the Minizinc IDE locally (recommended), or use their Minizinc web client. Minizinc is a human-readable language for describing constraint satisfaction problems (CSP). Minizinc also translates the CSP into a set of logical formulas and passes these to a SAT solver and translates the results back into a human-readable format.

Group Exercises (15P)

Exercise 1.b)

Implement a solution for the “super 512 queens problem”, i.e., an extension of the standard 8x8 chessboard to a 512x512 chessboard where each column has exactly one assigned queen and each queen also has “knight capabilities”, i.e., moving/behaving like a knight (in total, we have 512 “super queens”). If no solution could be found for 512 queens, find a solution that maximizes the number of super queens not attacking each other while keeping the chessboard size 512x512. Explain your implementation in detail (specifically, the way your algorithm generates populations, performs mutations, etc.). Develop your own genetic algorithm, i.e., do not use an existing library that supports such implementations. (10P)

Additional Constraints

- **Programming Languages**: We will accept Python, but absolutely no other languages. The program must be compilable without errors or warnings in python 3.12.
- **Libraries**: These Python libraries are explicitly whitelisted, but absolutely no other no other libraries are allowed: sys (and any other “basic” libraries), numpy, random, argparse, copy, numba.
- **Skeleton**: You have to use the provided python file 1b_skeleton.py for your implementation.
- **GUI**: If you want to use a GUI, make it optional (i.e. it runs only if started with some argument, e.g. --gui)

Additional Information

- We won't provide you with Gitlab repositories, but TU Graz provides Gitlab for every student. It's completely up to you how you organize your code.
- In your report DO NOT rephrase the exercise itself. Keep your explanations short and easy to read. Mention what approach you have taken to optimize your runtime, what worked, what didn't work. You can write in German or in English, whatever you prefer. Also don't let LLMs write your texts. The maximum length is 4 pages (for exercise 1.b).

General hints

- Before you start your implementation, think about how the following components for your GA are designed: Population Generation, Fitness Function, Selection Method, Crossover Operator, Mutation Operator, Termination Condition.
- There are multiple possibilities to encode an “individual”. Choose whatever turns out to be optimal in terms of computation.

Group Exercises

Files to Hand in:

- **report.pdf**: A PDF file containing your step-by-step execution of the backtracking with forward-checking algorithm, as well as the output of your Minizinc interpreter and your explanation of the implemented Genetic Algorithm.
- **1a.mzn**: A file containing your runnable Minizinc code
- **1b.py**: A file containing your implementation of the “super 512 Queens Problem”.
- **Only one team member must submit the assignment files and not zipped. This holds for all group assignments.**

Thank You!